01 LC 70 | Climbing Stairs (Easy)

Problem: You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Constraints: 1 <= n <= 45

```
class Solution {
public:
  int climbStairs(int n) {
    vector<vector<long long>> a = {{1, 1}, {1,
      0}};
    return pow(a, n - 1)[0][0];
  }
private:
  vector<vector<long long>> mul(vector<vector<long
    long>>& a, vector<vector<long long>>& b) {
    int m = a.size(), n = b[0].size();
    vector<vector<long long>> res(m, vector<long
      long>(n));
    for (int i = 0; i < m; ++i) {
      for (int j = 0; j < n; ++j) {
        for (int k = 0; k < a[0].size(); ++k) {
          res[i][j] += a[i][k] * b[k][j];
        }
      }
    }
    return res;
  }
  vector<vector<long long>> pow(vector<vector<long
    long>>& a, int n) {
    vector<vector<long long>> res = {{1, 1}, {0,
      0}};
    while (n) {
      if (n & 1) {
        res = mul(res, a);
      }
      a = mul(a, a);
      n >>= 1;
    }
    return res;
  }
};
```

25 LC 931 | Minimum Falling Path Sum (Medium)

Problem: Given an n x n array of integers matrix, return the minimum sum of any falling path through matrix. A falling path starts at any element in the first row and chooses the element in the next row that is either directly below or diagonally left/right. Specifically, the next element from position (row, col) will be (row + 1, col - 1), (row + 1, col), or (row + 1, col + 1).

Constraints: n == matrix.length == matrix[i].length; 1 <= n <= 100; -100 <= matrix[i][j] <= 100

```
class Solution {
public:
  int minFallingPathSum(vector<vector<int>>&
    matrix) {
    int n = matrix.size();
    vector<int> f(n);
    for (auto& row : matrix) {
      auto g = f;
      for (int j = 0; j < n; ++j) {
        if (j) {
          g[j] = min(g[j], f[j - 1]);
        }
        if (j + 1 < n) {
          g[j] = min(g[j], f[j + 1]);
        }
        g[j] += row[j];
      }
      f = move(g);
    }
    return *min_element(f.begin(), f.end());
  }
};
```

02 LC 198 | House Robber (Medium)

Problem: Return the maximum sum obtainable by robbing houses in a line without choosing adjacent houses. nums[i] is the amount in house i.

Constraints: 1 <= nums.length <= 100; 0 <= nums[i] <= 400

```
class Solution {
public:
  int rob(vector<int>& nums) {
    int f = 0, g = 0;
    for (int& x : nums) {
      int ff = max(f, g);
      g = f + x;
      f = ff;
    }
    return max(f, g);
  }
};
```

37 LC 714 | Best Time to Buy and Sell Stock with Transaction Fee (Medium)

Problem: Maximize stock profit with any number of transactions, holding at most one share. Each completed transaction incurs fee once. Return the maximum profit.

Constraints: 1 <= prices.length <= 5 * 10^4; 1 <= prices[i] < 5 * 10^4; 0 <= fee < 5 * 10^4

```
class Solution {
public:
  int maxProfit(vector<int>& prices, int fee) {
    int f0 = 0, f1 = -prices[0];
    for (int i = 1; i < prices.size(); ++i) {
      int g0 = max(f0, f1 + prices[i] - fee);
      f1 = max(f1, f0 - prices[i]);
      f0 = g0;
    }
    return f0;
  }
};
```

40 LC 121 | Best Time to Buy and Sell Stock (Easy)

Problem: Choose one day to buy and a later day to sell one stock. Return the maximum profit, or 0 when no profitable transaction exists.

Constraints: 1 <= prices.length <= 10^5; 0 <= prices[i] <= 10^4

```
class Solution {
public:
  int maxProfit(vector<int>& prices) {
    int ans = 0, mi = prices[0];
    for (int& v : prices) {
      ans = max(ans, v - mi);
      mi = min(mi, v);
    }
    return ans;
  }
};
```

42 LC 55 | Jump Game (Medium)

Problem: Starting at index 0, nums[i] is the maximum forward jump length from i. Return whether the last index can be reached.

Constraints: 1 <= nums.length <= 10^4; 0 <= nums[i] <= 10^5

```
class Solution {
public:
  bool canJump(vector<int>& nums) {
    int mx = 0;
    for (int i = 0; i < nums.size(); ++i) {
      if (mx < i) {
        return false;
      }
      mx = max(mx, i + nums[i]);
    }
    return true;
  }
};
```

03 LC 322 | Coin Change (Medium)
Problem: You are given an integer array coins representing coins of different denominations and an integer amount representing a total amount of money. Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return -1. You may assume that you have an infinite number of each kind of coin.
Constraints: 1 <= coins.length <= 12; 1 <= coins[i] <= 2^31 - 1; 0 <= amount <= 10^4

```
class Solution {
public:
  int coinChange(vector<int>& coins, int amount) {
    int n = amount;
    vector<int> f = vector<int>(n + 1,
      0x3f3f3f3f);
    f[0] = 0;
    for (int x : coins) {
      for (int j = x; j <= n; ++j) {
        f[j] = min(f[j], f[j - x] + 1);
      }
    }
    return f[n] > n ? -1 : f[n];
  }
};
```

07 LC 416 | Partition Equal Subset Sum (Medium)
Problem: Given an integer array nums, return true if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or false otherwise.
Constraints: 1 <= nums.length <= 200; 1 <= nums[i] <= 100

```
class Solution {
public:
  bool canPartition(vector<int>& nums) {
    int s = accumulate(nums.begin(), nums.end(),
      0);
    if (s % 2 == 1) {
      return false;
    }
    int m = s >> 1;
    vector<char> f = vector<char>(m + 1, 0);
    f[0] = true;
    for (int& x : nums) {
      for (int j = m; j >= x; --j) {
        f[j] |= f[j - x];
      }
    }
    return f[m];
  }
};
```

19 LC 518 | Coin Change II (Medium)
Problem: You are given an integer array coins representing coins of different denominations and an integer amount representing a total amount of money. Return the number of combinations that make up that amount. If that amount of money cannot be made up by any combination of the coins, return 0. You may assume that you have an infinite number of each kind of coin. The final answer is guaranteed to fit into a signed 32-bit integer.
Constraints: 1 <= coins.length <= 300; 1 <= coins[i] <= 5000; All the values of coins are unique.; 0 <= amount <= 5000

```
class Solution {
public:
  int change(int amount, vector<int>& coins) {
    vector<int> dp(amount + 1);
    dp[0] = 1;
    for (int coin : coins)
      for (int value = coin; value <= amount;
        ++value)
        dp[value] = static_cast<int>(min<long
          long>(INT_MAX, 1LL * dp[value] +
          dp[value - coin]));
    return dp[amount];
  }
};
```

04 LC 300 | Longest Increasing Subsequence (Medium)
Problem: Given an integer array nums, return the length of the longest strictly increasing subsequence.
Constraints: 1 <= nums.length <= 2500; -10^4 <= nums[i] <= 10^4

```
class Solution {
public:
  int lengthOfLIS(vector<int>& nums) {
    vector<int> tails;
    for (int x : nums) {
      auto it = ranges::lower_bound(tails, x);
      if (it == tails.end()) tails.push_back(x);
      else *it = x;
    }
    return tails.size();
  }
};
```

06 LC 72 | Edit Distance (Medium)
Problem: Given two strings word1 and word2, return the minimum number of operations required to convert word1 to word2. You have the following three operations permitted on a word: - Insert a character - Delete a character - Replace a character
Constraints: 0 <= word1.length, word2.length <= 500; word1 and word2 consist of lowercase English letters.

```
class Solution {
public:
  int minDistance(string a, string b) {
    if (a.size() < b.size()) swap(a, b);
    vector<int> dp(b.size() + 1);
    iota(dp.begin(), dp.end(), 0);
    for (int i = 1; i <=
      static_cast<int>(a.size()); ++i) {
      int diagonal = dp[0];
      dp[0] = i;
      for (int j = 1; j <=
        static_cast<int>(b.size()); ++j) {
        int above = dp[j];
        dp[j] = a[i - 1] == b[j - 1] ? diagonal :
          1 + min({diagonal, dp[j], dp[j - 1]});
        diagonal = above;
      }
    }
    return dp.back();
  }
};
```

49 LC 95 | Unique Binary Search Trees II (Medium)
Problem: Given an integer n, return all the structurally unique BST's (binary search trees), which has exactly n nodes of unique values from 1 to n. Return the answer in any order.
Constraints: 1 <= n <= 8

```
class Solution {
public:
  vector<TreeNode*> generateTrees(int n) {
    function<vector<TreeNode*>(int, int)> dfs =
      [&](int i, int j) {
      if (i > j) {
        return vector<TreeNode*>{nullptr};
      }
      vector<TreeNode*> ans;
      for (int v = i; v <= j; ++v) {
        auto left = dfs(i, v - 1);
        auto right = dfs(v + 1, j);
        for (auto l : left) {
          for (auto r : right) {
            ans.push_back(new TreeNode(v, l, r));
          }
        }
      }
      return ans;
    };
    return dfs(1, n);
  }
};
```

05 LC 1143 | Longest Common Subsequence (Medium)

Problem: Return the length of a longest common subsequence of text1 and text2. A subsequence may omit characters but must preserve their order; it need not be contiguous.
Constraints: 1 <= text1.length, text2.length <= 1000; text1 and text2 consist of only lowercase English characters.

```
class Solution {
public:
  int longestCommonSubsequence(string a, string b)
    {
    if (a.size() < b.size()) swap(a, b);
    vector<int> dp(b.size() + 1);
    for (char x : a) {
      int diagonal = 0;
      for (int j = 1; j <=
        static_cast<int>(b.size()); ++j) {
        int above = dp[j];
        dp[j] = x == b[j - 1] ? diagonal + 1 :
          max(dp[j], dp[j - 1]);
        diagonal = above;
      }
    }
    return dp.back();
  }
};
```

14 LC 746 | Min Cost Climbing Stairs (Easy)

Problem: You are given an integer array cost where cost[i] is the cost of ith step on a staircase. Once you pay the cost, you can either climb one or two steps. You can either start from the step with index 0, or the step with index 1. Return the minimum cost to reach the top of the staircase, which is the position just past the last step (index cost.length).
Constraints: 2 <= cost.length <= 1000; 0 <= cost[i] <= 999

```
class Solution {
public:
  int minCostClimbingStairs(vector<int>& cost) {
    int f = 0, g = 0;
    for (int i = 2; i <= cost.size(); ++i) {
      int gg = min(f + cost[i - 2], g + cost[i -
        1]);
      f = g;
      g = gg;
    }
    return g;
  }
};
```

33 LC 312 | Burst Balloons (Hard)

Problem: Burst every balloon to maximize total coins. Bursting balloon i earns the product of its current left neighbor, itself, and its current right neighbor; missing neighbors have value 1.
Constraints: n == nums.length; 1 <= n <= 300; 0 <= nums[i] <= 100

```
class Solution {
public:
  int maxCoins(vector<int>& nums) {
    int n = nums.size();
    vector<int> arr(n + 2, 1);
    for (int i = 0; i < n; ++i) {
      arr[i + 1] = nums[i];
    }
    vector<vector<int>> f(n + 2, vector<int>(n +
      2, 0));
    for (int i = n - 1; i >= 0; --i) {
      for (int j = i + 2; j <= n + 1; ++j) {
        for (int k = i + 1; k < j; ++k) {
          f[i][j] = max(f[i][j], f[i][k] + f[k][j]
            + arr[i] * arr[k] * arr[j]);
        }
      }
    }
    return f[0][n + 1];
  }
};
```

08 LC 139 | Word Break (Medium)

Problem: Given a string s and a dictionary of strings wordDict, return true if s can be segmented into a space-separated sequence of one or more dictionary words. Note that the same word in the dictionary may be reused multiple times in the segmentation.
Constraints: 1 <= s.length <= 300; 1 <= wordDict.length <= 1000; 1 <= wordDict[i].length <= 20; s and wordDict[i] consist of only lowercase English letters.; All the strings of wordDict are unique.

```
class Solution {
  struct TrieNode { array<int, 26> next; bool
    terminal = false; TrieNode() { next.fill(-1);
    } };
public:
  bool wordBreak(string s, vector<string>&
    wordDict) {
    vector<TrieNode> trie(1);
    for (const auto& word : wordDict) {
      int node = 0;
      for (char c : word) {
        int letter = c - 'a';
        if (trie[node].next[letter] == -1) {
          int child = trie.size();
          trie.emplace_back();
          trie[node].next[letter] = child;
        }
        node = trie[node].next[letter];
      }
      trie[node].terminal = true;
    }
    vector<bool> reachable(s.size() + 1);
    reachable[0] = true;
    for (int start = 0; start <
      static_cast<int>(s.size()); ++start) {
      if (!reachable[start]) continue;
      int node = 0;
      for (int end = start; end <
        static_cast<int>(s.size()); ++end) {
        node = trie[node].next[s[end] - 'a'];
        if (node == -1) break;
        if (trie[node].terminal) reachable[end +
          1] = true;
      }
    }
    return reachable.back();
  }
};
```

20 LC 494 | Target Sum (Medium)

Problem: Place a '+' or '-' before every nums element and add the signed values. Return the number of assignments whose result equals target; each array occurrence is a separate choice.
Constraints: 1 <= nums.length <= 20; 0 <= nums[i] <= 1000; 0 <= sum(nums[i]) <= 1000; -1000 <= target <= 1000

```
class Solution {
public:
  int findTargetSumWays(vector<int>& nums, int
    target) {
    int s = accumulate(nums.begin(), nums.end(),
      0);
    if (s < target || (s - target) % 2) {
      return 0;
    }
    int n = (s - target) / 2;
    vector<int> f = vector<int>(n + 1, 0);
    f[0] = 1;
    for (int x : nums) {
      for (int j = n; j >= x; --j) {
        f[j] += f[j - x];
      }
    }
    return f[n];
  }
};
```

09 LC 62 | Unique Paths (Medium)
Problem: A robot starts at the top-left of an m-by-n grid and moves only right or down. Return the number of paths to the bottom-right.
Constraints: 1 <= m, n <= 100

```
class Solution {
public:
  int uniquePaths(int m, int n) {
    int k = min(m - 1, n - 1), total = m + n - 2;
    long long paths = 1;
    for (int i = 1; i <= k; ++i) paths = paths *
      (total - k + i) / i;
    return static_cast<int>(paths);
  }
};
```

11 LC 53 | Maximum Subarray (Medium)
Problem: Return the largest sum of a nonempty contiguous subarray of nums.
Constraints: 1 <= nums.length <= 10^5; -10^4 <= nums[i] <= 10^4

```
class Solution {
public:
  int maxSubArray(vector<int>& nums) {
    int ans = nums[0], f = nums[0];
    for (int i = 1; i < nums.size(); ++i) {
      f = max(f, 0) + nums[i];
      ans = max(ans, f);
    }
    return ans;
  }
};
```

41 LC 122 | Best Time to Buy and Sell Stock II (Medium)
Problem: Maximize stock profit with any number of buy/sell transactions, holding at most one share at a time. Buying and selling on the same day is allowed; return the maximum profit.
Constraints: 1 <= prices.length <= 3 * 10^4; 0 <= prices[i] <= 10^4

```
class Solution {
public:
  int maxProfit(vector<int>& prices) {
    int ans = 0;
    for (int i = 1; i < prices.size(); ++i) ans +=
      max(0, prices[i] - prices[i - 1]);
    return ans;
  }
};
```

48 LC 96 | Unique Binary Search Trees (Medium)
Problem: Given an integer n, return the number of structurally unique BST's (binary search trees) which has exactly n nodes of unique values from 1 to n.
Constraints: 1 <= n <= 19

```
class Solution {
public:
  int numTrees(int n) {
    long long count = 1;
    for (int i = 0; i < n; ++i) count = count * 2
      * (2 * i + 1) / (i + 2);
    return static_cast<int>(count);
  }
};
```

10 LC 64 | Minimum Path Sum (Medium)
Problem: Given a m x n grid filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path. Note: You can only move either down or right at any point in time.
Constraints: m == grid.length; n == grid[i].length; 1 <= m, n <= 200; 0 <= grid[i][j] <= 200

```
class Solution {
public:
  int minPathSum(vector<vector<int>>& grid) {
    vector<int> dp(grid[0].size(), INT_MAX / 2);
    dp[0] = 0;
    for (const auto& row : grid)
      for (int j = 0; j <
        static_cast<int>(row.size()); ++j)
        dp[j] = row[j] + min(dp[j], j ? dp[j - 1]
          : INT_MAX / 2);
    return dp.back();
  }
};
```

38 LC 123 | Best Time to Buy and Sell Stock III (Hard)
Problem: You are given an array prices where prices[i] is the price of a given stock on the ith day. Find the maximum profit you can achieve. You may complete at most two transactions. Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).
Constraints: 1 <= prices.length <= 10^5; 0 <= prices[i] <= 10^5

```
class Solution {
public:
  int maxProfit(vector<int>& prices) {
    int f1 = -prices[0], f2 = 0, f3 = -prices[0],
      f4 = 0;
    for (int i = 1; i < prices.size(); ++i) {
      f1 = max(f1, -prices[i]);
      f2 = max(f2, f1 + prices[i]);
      f3 = max(f3, f2 - prices[i]);
      f4 = max(f4, f3 + prices[i]);
    }
    return f4;
  }
};
```

43 LC 45 | Jump Game II (Medium)
Problem: Starting at index 0, jump forward by at most nums[i] positions from index i. Return the minimum jumps needed to reach the last index, which is guaranteed reachable.
Constraints: 1 <= nums.length <= 10^4; 0 <= nums[i] <= 1000; It's guaranteed that you can reach nums[n - 1].

```
class Solution {
public:
  int jump(vector<int>& nums) {
    int ans = 0, mx = 0, last = 0;
    for (int i = 0; i < nums.size() - 1; ++i) {
      mx = max(mx, i + nums[i]);
      if (last == i) {
        ++ans;
        last = mx;
      }
    }
    return ans;
  }
};
```

12 LC 152 | Maximum Product Subarray (Medium)
Problem: Return the largest product of a nonempty contiguous subarray of nums. A one-element subarray has that element's value as its product.
Constraints: 1 <= nums.length <= 2 * 10^4; -10 <= nums[i] <= 10; The product of any subarray of nums is guaranteed to fit in a 32-bit integer.

```
class Solution {
public:
  int maxProduct(vector<int>& nums) {
    int f = nums[0], g = nums[0], ans = nums[0];
    for (int i = 1; i < nums.size(); ++i) {
      int ff = f, gg = g;
      f = max({nums[i], ff * nums[i], gg *
        nums[i]});
      g = min({nums[i], ff * nums[i], gg *
        nums[i]});
      ans = max(ans, f);
    }
    return ans;
  }
};
```

15 LC 213 | House Robber II (Medium)
Problem: Return the maximum amount robbed from houses arranged in a circle without robbing adjacent houses. The first and last houses are neighbors.
Constraints: 1 <= nums.length <= 100; 0 <= nums[i] <= 1000

```
class Solution {
public:
  int rob(vector<int>& nums) {
    int n = nums.size();
    if (n == 1) {
      return nums[0];
    }
    return max(robRange(nums, 0, n - 2),
      robRange(nums, 1, n - 1));
  }
  int robRange(vector<int>& nums, int l, int r) {
    int f = 0, g = 0;
    for (; l <= r; ++l) {
      int ff = max(f, g);
      g = f + nums[l];
      f = ff;
    }
    return max(f, g);
  }
};
```

26 LC 63 | Unique Paths II (Medium)
Problem: In obstacleGrid, 1 marks a blocked cell and 0 an open cell. Count paths from top-left to bottom-right using only right/down moves and no blocked cells. The answer fits signed 32-bit int.
Constraints: m == obstacleGrid.length; n == obstacleGrid[i].length; 1 <= m, n <= 100; obstacleGrid[i][j] is 0 or 1.

```
class Solution {
public:
  int
    uniquePathsWithObstacles(vector<vector<int>>&
    grid) {
    vector<int> dp(grid[0].size());
    dp[0] = 1;
    for (const auto& row : grid) {
      for (int j = 0; j <
        static_cast<int>(row.size()); ++j) {
        if (row[j]) dp[j] = 0;
        else if (j) dp[j] =
          static_cast<int>(min<long long>(INT_MAX,
          1LL * dp[j] + dp[j - 1]));
      }
    }
    return dp.back();
  }
};
```

13 LC 91 | Decode Ways (Medium)
Problem: Decode digit string s using 1=A through 26=Z. Return the number of complete decodings; tokens with a leading zero are invalid. The answer fits signed 32-bit int.
Constraints: 1 <= s.length <= 100; s contains only digits and may contain leading zero(s).

```
class Solution {
public:
  int numDecodings(string s) {
    int n = s.size();
    int f = 0, g = 1;
    for (int i = 1; i <= n; ++i) {
      int h = s[i - 1] != '0' ? g : 0;
      if (i > 1 && (s[i - 2] == '1' || (s[i - 2]
        == '2' && s[i - 1] <= '6'))) {
        h += f;
      }
      f = g;
      g = h;
    }
    return g;
  }
};
```

16 LC 337 | House Robber III (Medium)
Problem: Tree nodes represent houses with money. Return the maximum sum robbed without selecting both endpoints of any parent-child edge.
Constraints: The number of nodes in the tree is in the range [1, 10^4].; 0 <= Node.val <= 10^4

```
class Solution {
public:
  int rob(TreeNode* root) {
    function<pair<int, int>(TreeNode*)> dfs =
      [&](TreeNode* root) -> pair<int, int> {
      if (!root) {
        return make_pair(0, 0);
      }
      auto [la, lb] = dfs(root->left);
      auto [ra, rb] = dfs(root->right);
      return make_pair(root->val + lb + rb,
        max(la, lb) + max(ra, rb));
    };
    auto [a, b] = dfs(root);
    return max(a, b);
  }
};
```

23 LC 221 | Maximal Square (Medium)
Problem: Given an m x n binary matrix filled with 0's and 1's, find the largest square containing only 1's and return its area.
Constraints: m == matrix.length; n == matrix[i].length; 1 <= m, n <= 300; matrix[i][j] is '0' or '1'.

```
class Solution {
public:
  int maximalSquare(vector<vector<char>>& matrix)
    {
    vector<int> dp(matrix[0].size() + 1);
    int best = 0;
    for (const auto& row : matrix) {
      int diagonal = 0;
      for (int j = 1; j <=
        static_cast<int>(row.size()); ++j) {
        int above = dp[j];
        dp[j] = row[j - 1] == '1' ? 1 +
          min({dp[j], dp[j - 1], diagonal}) : 0;
        best = max(best, dp[j]); diagonal = above;
      }
    }
    return best * best;
  }
};
```

17 LC 279 | Perfect Squares (Medium)

Problem: Given an integer n, return the least number of perfect square numbers that sum to n. A perfect square is an integer that is the square of an integer; in other words, it is the product of some integer with itself. For example, 1, 4, 9, and 16 are perfect squares while 3 and 11 are not.

Constraints: 1 <= n <= 10^4

```
class Solution {
  bool square(int n) {
    int root = static_cast<int>(sqrt(n));
    return root * root == n;
  }
public:
  int numSquares(int n) {
    if (square(n)) return 1;
    int reduced = n;
    while (reduced % 4 == 0) reduced /= 4;
    if (reduced % 8 == 7) return 4;
    for (int x = 1; x * x <= n; ++x) if (square(n
      - x * x)) return 2;
    return 3;
  }
};
```

24 LC 120 | Triangle (Medium)

Problem: Given a triangle array, return the minimum path sum from top to bottom. For each step, you may move to an adjacent number of the row below. More formally, if you are on index i on the current row, you may move to either index i or index i + 1 on the next row.

Constraints: 1 <= triangle.length <= 200; triangle[0].length == 1; triangle[i].length == triangle[i - 1].length + 1; -10^4 <= triangle[i][j] <= 10^4

```
class Solution {
public:
  int minimumTotal(vector<vector<int>>& triangle)
    {
    int n = triangle.size();
    vector<int> f(n + 1, 0);
    for (int i = n - 1; i >= 0; --i) {
      for (int j = 0; j <= i; ++j) {
        f[j] = min(f[j], f[j + 1]) +
          triangle[i][j];
      }
    }
    return f[0];
  }
};
```

35 LC 1039 | Minimum Score Triangulation of Polygon (Medium)

Problem: A convex polygon has vertex weights values in cyclic order. Triangulate it using noncrossing diagonals; each triangle costs the product of its three vertex weights. Return the minimum total triangulation score.

Constraints: n == values.length; 3 <= n <= 50; 1 <= values[i] <= 100

```
class Solution {
public:
  int minScoreTriangulation(vector<int>& values) {
    int n = values.size();
    vector<vector<int>> f(n, vector<int>(n));
    for (int i = n - 3; i >= 0; --i) {
      for (int j = i + 2; j < n; ++j) {
        f[i][j] = 1 << 30;
        for (int k = i + 1; k < j; ++k) {
          f[i][j] = min(f[i][j], f[i][k] + f[k][j]
            + values[i] * values[k] * values[j]);
        }
      }
    }
    return f[0][n - 1];
  }
};
```

18 LC 377 | Combination Sum IV (Medium)

Problem: Given an array of distinct integers nums and a target integer target, return the number of possible combinations that add up to target. The test cases are generated so that the answer can fit in a 32-bit integer.

Constraints: 1 <= nums.length <= 200; 1 <= nums[i] <= 1000; All the elements of nums are unique.; 1 <= target <= 1000

```
class Solution {
public:
  int combinationSum4(vector<int>& nums, int
    target) {
    vector<int> dp(target + 1);
    dp[0] = 1;
    for (int sum = 1; sum <= target; ++sum)
      for (int x : nums) if (x <= sum)
        dp[sum] = static_cast<int>(min<long
          long>(INT_MAX, 1LL * dp[sum] + dp[sum -
          x]));
    return dp[target];
  }
};
```

31 LC 516 | Longest Palindromic Subsequence (Medium)

Problem: Given a string s, find the longest palindromic subsequence's length in s. A subsequence is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

Constraints: 1 <= s.length <= 1000; s consists only of lowercase English letters.

```
class Solution {
public:
  int longestPalindromeSubseq(string s) {
    int n = s.size();
    vector<int> dp(n);
    for (int i = n - 1; i >= 0; --i) {
      int diagonal = 0;
      dp[i] = 1;
      for (int j = i + 1; j < n; ++j) {
        int old = dp[j];
        dp[j] = s[i] == s[j] ? diagonal + 2 :
          max(dp[j], dp[j - 1]);
        diagonal = old;
      }
    }
    return dp.back();
  }
};
```

32 LC 1312 | Minimum Insertion Steps to Make a String Palindrome (Hard)

Problem: Given a string s. In one step you can insert any character at any index of the string. Return the minimum number of steps to make s palindrome. A Palindrome String is one that reads the same backward as well as forward.

Constraints: 1 <= s.length <= 500; s consists of lowercase English letters.

```
class Solution {
public:
  int minInsertions(string s) {
    int n = s.size();
    vector<int> dp(n);
    for (int i = n - 2; i >= 0; --i) {
      int diagonal = 0;
      for (int j = i + 1; j < n; ++j) {
        int old = dp[j];
        dp[j] = s[i] == s[j] ? diagonal : 1 +
          min(dp[j], dp[j - 1]);
        diagonal = old;
      }
    }
    return dp.back();
  }
};
```

21 LC 474 | Ones and Zeroes (Medium)

Problem: You are given an array of binary strings strs and two integers m and n. Return the size of the largest subset of strs such that there are at most m 0's and n 1's in the subset. A set x is a subset of a set y if all elements of x are also elements of y.
Constraints: 1 <= strs.length <= 600; 1 <= strs[i].length <= 100; strs[i] consists only of digits '0' and '1'.; 1 <= m, n <= 100

```
class Solution {
public:
  int findMaxForm(vector<string>& strs, int m, int
    n) {
    vector<vector<int>> f = vector<vector<int>>(m
      + 1, vector<int>(n + 1, 0));
    for (auto& s : strs) {
      auto [a, b] = count(s);
      for (int i = m; i >= a; --i) {
        for (int j = n; j >= b; --j) {
          f[i][j] = max(f[i][j], f[i - a][j - b] +
            1);
        }
      }
    }
    return f[m][n];
  }
  pair<int, int> count(string& s) {
    int a = count_if(s.begin(), s.end(), [](char
      c) { return c == '0'; });
    return {a, s.size() - a};
  }
};
```

46 LC 1235 | Maximum Profit in Job Scheduling (Hard)

Problem: We have n jobs, where every job is scheduled to be done from startTime[i] to endTime[i], obtaining a profit of profit[i]. You're given the startTime, endTime and profit arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range. If you choose a job that ends at time X you will be able to start another job that starts at time X.
Constraints: 1 <= startTime.length == endTime.length == profit.length <= 5 * 10^4; 1 <= startTime[i] < endTime[i] <= 10^9; 1 <= profit[i] <= 10^4

```
class Solution {
public:
  int jobScheduling(vector<int>& startTime,
    vector<int>& endTime, vector<int>& profit) {
    int n = profit.size();
    vector<tuple<int, int, int>> jobs(n);
    for (int i = 0; i < n; ++i) jobs[i] =
      {endTime[i], startTime[i], profit[i]};
    ranges::sort(jobs);
    vector<int> dp(n + 1);
    for (int i = 0; i < n; ++i) {
      auto [_, s, p] = jobs[i];
      int j = upper_bound(jobs.begin(),
        jobs.begin() + i, s, [&](int x, auto& job)
        -> bool { return x < get<0>(job); }) -
        jobs.begin();
      dp[i + 1] = max(dp[i], dp[j] + p);
    }
    return dp[n];
  }
};
```

22 LC 1049 | Last Stone Weight II (Medium)

Problem: Choose any pair of stones and replace them by their nonnegative weight difference, discarding a zero difference. Continue until at most one remains. Return the smallest achievable final weight, or 0 if none remains.
Constraints: 1 <= stones.length <= 30; 1 <= stones[i] <= 100

```
class Solution {
public:
  int lastStoneWeightII(vector<int>& stones) {
    int s = accumulate(stones.begin(),
      stones.end(), 0);
    int n = s >> 1;
    vector<int> dp(n + 1);
    for (int& v : stones)
      for (int j = n; j >= v; --j)
        dp[j] = max(dp[j], dp[j - v] + v);
    return s - dp[n] * 2;
  }
};
```

27 LC 115 | Distinct Subsequences (Hard)

Problem: Given two strings s and t, return the number of distinct subsequences of s which equals t. The test cases are generated so that the answer fits on a 32-bit signed integer.
Constraints: 1 <= s.length, t.length <= 1000; s and t consist of English letters.

```
class Solution {
public:
  int numDistinct(string s, string t) {
    vector<int> dp(t.size() + 1);
    dp[0] = 1;
    for (char c : s)
      for (int j = t.size(); j >= 1; --j)
        if (c == t[j - 1]) dp[j] =
          static_cast<int>(min<long long>(INT_MAX,
          1LL * dp[j] + dp[j - 1]));
    return dp.back();
  }
};
```

36 LC 309 l Best Time to Buv and Sell Stock with Cooldown (Medium)

Problem: Maximize profit with unlimited stock transactions and at most one share held. After selling, wait one full day before buying again. Return the maximum total profit.
Constraints: 1 <= prices.length <= 5000; 0 <= prices[i] <= 1000

```
class Solution {
public:
  int maxProfit(vector<int>& prices) {
    int f = 0, f0 = 0, f1 = -prices[0];
    for (int i = 1; i < prices.size(); ++i) {
      int g0 = max(f0, f1 + prices[i]);
      f1 = max(f1, f - prices[i]);
      f = f0;
      f0 = g0;
    }
    return f0;
  }
};
```

28 LC 97 | Interleaving String (Medium)
Problem: Return whether s3 can be formed by interleaving all characters of s1 and s2 while preserving each source string's internal order.
Constraints: 0 <= s1.length, s2.length <= 100; 0 <= s3.length <= 200; s1, s2, and s3 consist of lowercase English letters.

```
class Solution {
public:
  bool isInterleave(string s1, string s2, string
    s3) {
    int m = s1.size(), n = s2.size();
    if (m + n != s3.size()) {
      return false;
    }
    vector<char> f = vector<char>(n + 1, 0);
    f[0] = true;
    for (int i = 0; i <= m; ++i) {
      for (int j = 0; j <= n; ++j) {
        int k = i + j - 1;
        if (i) {
          f[j] &= s1[i - 1] == s3[k];
        }
        if (j) {
          f[j] |= (s2[j - 1] == s3[k] && f[j -
            1]);
        }
      }
    }
    return f[n];
  }
};
```

50 LC 10 | Regular Expression Matching (Hard)
Problem: Given an input string s and a pattern p, implement regular expression matching with support for '.' and '*' where: - '.' Matches any single character. - '*' Matches zero or more of the preceding element. Return a boolean indicating whether the matching covers the entire input string (not partial).
Constraints: 1 <= s.length <= 20; 1 <= p.length <= 20; s contains only lowercase English letters.; p contains only lowercase English letters, '.', and '*'.; It is guaranteed for each appearance of the character '*', there will be a previous valid character to match.

```
class Solution {
public:
  bool isMatch(string s, string p) {
    int n = p.size();
    vector<bool> previous(n + 1), current(n + 1);
    previous[0] = true;
    for (int j = 2; j <= n; ++j) if (p[j - 1] ==
      '*') previous[j] = previous[j - 2];
    for (char c : s) {
      fill(current.begin(), current.end(), false);
      for (int j = 1; j <= n; ++j) {
        if (p[j - 1] == '*') {
          bool matches = p[j - 2] == '.' || p[j -
            2] == c;
          current[j] = current[j - 2] || (matches
            && previous[j]);
        } else current[j] = previous[j - 1] &&
          (p[j - 1] == '.' || p[j - 1] == c);
      }
      previous.swap(current);
    }
    return previous[n];
  }
};
```

29 LC 5 | Longest Palindromic Substring (Medium)
Problem: Return a longest contiguous palindromic substring of s. Any longest answer is acceptable.
Constraints: 1 <= s.length <= 1000; s consist of only digits and English letters.

```
class Solution {
  vector<int> radii(const string& s) {
    int m = 2 * static_cast<int>(s.size()) + 1;
    vector<int> p(m);
    auto equal = [&](int a, int b) {
      return a % 2 == b % 2 && (a % 2 == 0 || s[a
        / 2] == s[b / 2]);
    };
    int center = 0, right = 0;
    for (int i = 0; i < m; ++i) {
      if (i < right) p[i] = min(right - i, p[2 *
        center - i]);
      while (i - p[i] - 1 >= 0 && i + p[i] + 1 < m
        &&
         equal(i - p[i] - 1, i + p[i] + 1))
          ++p[i];
      if (i + p[i] > right) { center = i; right =
        i + p[i]; }
    }
    return p;
  }
public:
  string longestPalindrome(string s) {
    auto p = radii(s);
    int center = ranges::max_element(p) -
      p.begin();
    return s.substr((center - p[center]) / 2,
      p[center]);
  }
};
```

45 LC 354 | Russian Doll Envelopes (Hard)
Problem: You are given a 2D array of integers envelopes where envelopes[i] = [w_i, h_i] represents the width and the height of an envelope. One envelope can fit into another if and only if both the width and height of one envelope are greater than the other envelope's width and height. Return the maximum number of envelopes you can Russian doll (i.e., put one inside the other). Note: You cannot rotate an envelope.
Constraints: 1 <= envelopes.length <= 10^5; envelopes[i].length == 2; 1 <= w_i, h_i <= 10^5

```
class Solution {
public:
  int maxEnvelopes(vector<vector<int>>& envelopes)
    {
    ranges::sort(envelopes, [](const auto& e1,
      const auto& e2) {
      return e1[0] < e2[0] || (e1[0] == e2[0] &&
        e1[1] > e2[1]);
    });
    int n = envelopes.size();
    vector<int> d{envelopes[0][1]};
    for (int i = 1; i < n; ++i) {
      int x = envelopes[i][1];
      if (x > d[d.size() - 1])
        d.push_back(x);
      else {
        int idx = lower_bound(d.begin(), d.end(),
          x) - d.begin();
        if (idx == d.size()) idx = 0;
        d[idx] = x;
      }
    }
    return d.size();
  }
};
```

30 LC 647 | Palindromic Substrings (Medium)

Problem: Given a string s, return the number of palindromic substrings in it. A string is a palindrome when it reads the same backward as forward. A substring is a contiguous sequence of characters within the string.
Constraints: 1 <= s.length <= 1000; s consists of lowercase English letters.

```
class Solution {
  vector<int> radii(const string& s) {
    int m = 2 * static_cast<int>(s.size()) + 1;
    vector<int> p(m);
    auto equal = [&](int a, int b) {
      return a % 2 == b % 2 && (a % 2 == 0 || s[a
        / 2] == s[b / 2]);
    };
    int center = 0, right = 0;
    for (int i = 0; i < m; ++i) {
      if (i < right) p[i] = min(right - i, p[2 *
        center - i]);
      while (i - p[i] - 1 >= 0 && i + p[i] + 1 < m
        &&
         equal(i - p[i] - 1, i + p[i] + 1))
          ++p[i];
      if (i + p[i] > right) { center = i; right =
        i + p[i]; }
    }
    return p;
  }
public:
  int countSubstrings(string s) {
    auto p = radii(s);
    int answer = 0;
    for (int radius : p) answer += (radius + 1) /
      2;
    return answer;
  }
};
```

34 LC 1547 | Minimum Cost to Cut a Stick (Hard)

Problem: Cut a stick of length n at all positions in cuts, in any order. Cutting a current piece costs that piece's length. Return the minimum total cost.
Constraints: 2 <= n <= 10^6; 1 <= cuts.length <= min(n - 1, 100); 1 <= cuts[i] <= n - 1; All the integers in cuts array are distinct.

```
class Solution {
public:
  int minCost(int n, vector<int>& cuts) {
    vector<int> position = cuts;
    position.push_back(0); position.push_back(n);
    ranges::sort(position);
    int size = position.size();
    vector<vector<int>> dp(size,
      vector<int>(size));
    vector<vector<int>> optimum(size,
      vector<int>(size));
    for (int i = 0; i + 1 < size; ++i)
      optimum[i][i + 1] = i + 1;
    for (int gap = 2; gap < size; ++gap) {
      for (int i = 0; i + gap < size; ++i) {
        int j = i + gap;
        dp[i][j] = INT_MAX;
        int lo = max(i + 1, optimum[i][j - 1]);
        int hi = min(j - 1, optimum[i + 1][j]);
        for (int k = lo; k <= hi; ++k) {
          int value = dp[i][k] + dp[k][j] +
            position[j] - position[i];
          if (value < dp[i][j]) { dp[i][j] =
            value; optimum[i][j] = k; }
        }
      }
    }
    return dp[0][size - 1];
  }
};
```

39 LC 188 | Best Time to Buy and Sell Stock IV (Hard)

Problem: You are given an integer array prices where prices[i] is the price of a given stock on the ith day, and an integer k. Find the maximum profit you can achieve. You may complete at most k transactions: i.e. you may buy at most k times and sell at most k times. Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).
Constraints: 1 <= k <= 100; 1 <= prices.length <= 1000; 0 <= prices[i] <= 1000

```
class Solution {
public:
  int maxProfit(int k, vector<int>& prices) {
    int n = prices.size();
    vector<vector<int>> f = vector<vector<int>>(k
      + 1, vector<int>(2, 0));
    for (int j = 1; j <= k; ++j) {
      f[j][1] = -prices[0];
    }
    for (int i = 1; i < n; ++i) {
      for (int j = k; j; --j) {
        f[j][0] = max(f[j][1] + prices[i],
          f[j][0]);
        f[j][1] = max(f[j - 1][0] - prices[i],
          f[j][1]);
      }
    }
    return f[k][0];
  }
};
```

47 LC 983 | Minimum Cost For Tickets (Medium)

Problem: Travel is required on the listed days in a 365-day year. Passes cost costs[0..2] and cover 1, 7, or 30 consecutive days starting at purchase. Return the minimum cost covering all travel days.
Constraints: 1 <= days.length <= 365; 1 <= days[i] <= 365; days is in strictly increasing order.; costs.length == 3; 1 <= costs[i] <= 1000

```
class Solution {
public:
  int mincostTickets(vector<int>& days,
    vector<int>& costs) {
    int m = days.back();
    vector<int> f = vector<int>(m + 1, 0);
    f[0] = 0;
    int valid[3] = {1, 7, 30};
    for (int i = 1, j = 0; i <= m; ++i) {
      if (i == days[j]) {
        f[i] = INT_MAX;
        for (int k = 0; k < 3; ++k) {
          int c = costs[k], v = valid[k];
          f[i] = min(f[i], f[max(0, i - v)] + c);
        }
        ++j;
      } else {
        f[i] = f[i - 1];
      }
    }
    return f[m];
  }
};
```

44 LC 673 | Number of Longest Increasing Subsequence (Medium)

Problem: Given an integer array nums, return the number of longest increasing subsequences. Notice that the sequence has to be strictly increasing.
Constraints: 1 <= nums.length <= 2000; -10^6 <= nums[i] <= 10^6; The answer is guaranteed to fit inside a 32-bit integer.

```
class BinaryIndexedTree {
private:
  int n;
  vector<int> c;
  vector<int> d;
public:
  BinaryIndexedTree(int n)
    : n(n)
    , c(n + 1, 0)
    , d(n + 1, 0) {}
  void update(int x, int v, int cnt) {
    while (x <= n) {
      if (c[x] < v) {
        c[x] = v;
        d[x] = cnt;
      } else if (c[x] == v) {
        d[x] += cnt;
      }
      x += x & -x;
    }
  }
  pair<int, int> query(int x) {
    int v = 0, cnt = 0;
    while (x > 0) {
      if (c[x] > v) {
        v = c[x];
        cnt = d[x];
      } else if (c[x] == v) {
        cnt += d[x];
      }
      x -= x & -x;
    }
    return {v, cnt};
  }
};
class Solution {
public:
  int findNumberOfLIS(vector<int>& nums) {
    vector<int> arr = nums;
    ranges::sort(arr);
    arr.erase(unique(arr.begin(), arr.end()),
      arr.end());
    int m = arr.size();
    BinaryIndexedTree tree(m);
    for (int x : nums) {
      auto it = lower_bound(arr.begin(),
        arr.end(), x);
      int i = distance(arr.begin(), it) + 1;
      auto [v, cnt] = tree.query(i - 1);
      tree.update(i, v + 1, max(cnt, 1));
    }
    return tree.query(m).second;
  }
};
```